# Développer avec GitHub Copilot

## OBJECTIFS

- Situer Copilot comme le middleware entre l'éditeur et le modèle.
- Écrire le code qui reste : commentaires, complétion, Ctrl+I.
- Réserver l'agent aux tests unitaires et aux POC.
- Lire une requête : tokens, historique, RAG, fonctions, outils.
- Injecter une instruction, un agent ou un skill au moment où il part.

## DÉROULÉ

| MOMENT | SUJET |
|---|---|
| 1 | Qu'est-ce que Copilot |
| 2 | Le chat et le produit stable |
| 3 | Commentaires, complétion, Ctrl+I |
| 4 | Mise en pratique des trois gestes |
| 5 | Ce qui part dans la requête |
| 6 | Qu'est-ce qu'un agent |
| 7 | Les configurations |
| 8 | Mise en pratique : un skill, un POC |

## VOCABULAIRE

| MOT | SENS |
|---|---|
| Chat | Le coding agentique. L'agent écrit le code |
| Ask et Ctrl+I | Ne s'appellent pas chat |

## DÉVELOPPER AVEC GITHUB COPILOT

# Introduction

**DÉFINITION — CHAT**

L'agent qui écrit le code. Ask et Ctrl+I ne s'appellent pas chat.

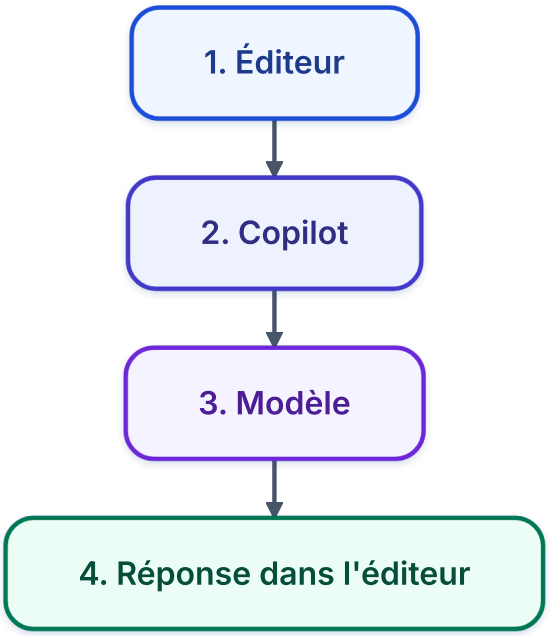
- Présentation : About GitHub Copilot
- Personnaliser : Response customization

# Module 1 — Qu'est-ce que Copilot

Copilot n'est pas le modèle.

**DÉFINITION — MIDDLEWARE**

Couche entre l'éditeur et le modèle. Elle fabrique la requête. Elle l'envoie. Elle réinjecte la réponse.




- L'éditeur donne le curseur et les fichiers ouverts.
- Le modèle répond à ce texte seul.
- Le modèle ne voit pas le projet.
- Le modèle ne voit pas les fichiers fermés.
- Le modèle peut changer. Cette couche reste.
- Un extrait copié entre dans la requête.

### LEXIQUE

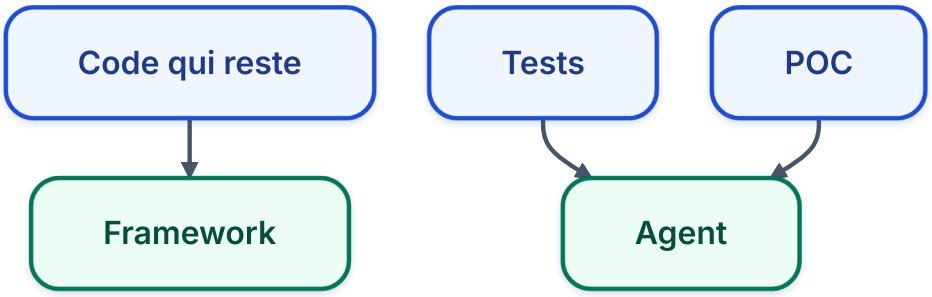
| TERME | SIGNIFICATION |
|---|---|
| Copilot | Le middleware. Pas le modèle |
| Modèle | Lit la requête. Produit la réponse |
| Requête | Le seul texte que le modèle reçoit |
| Chat | L'agent qui écrit le code |

# Module 2 — Le chat et le produit stable

**DÉFINITION — CHAT**

L'agent qui écrit le code. Ask et Ctrl+I ne s'appellent pas chat.

- Le code qui reste ne s'écrit pas dans le chat.
- La qualité et la stabilité comptent.
- Ce qui n'est pas écrit est tranché en silence.
- La connaissance de la codebase se perd.
- Le framework fait déjà gagner du temps.
- Commentaires. Puis complétion. Puis Ctrl+I.
- Un code de qualité est plus rapide ainsi. Il est aussi moins coûteux.



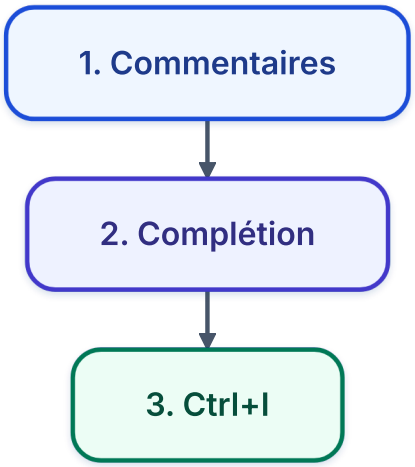

- L'agent écrit les tests unitaires.
- L'agent fait un POC.
- L'agent n'écrit pas le code qui reste.

## MESURES

| SOURCE | CONSTAT |
|---|---|
| Ford, TechSparks 2025 | Couverture 85 %. Plus de 2 000 tests pour 40 lignes. Build de 10 à 25 minutes. Code redondant en production. La revue humaine revient |
| CodeRabbit, 470 PR | 10,83 problèmes par PR côté IA. 6,45 côté humain |
| Veracode, 2025 | Faille du Top 10 OWASP dans 45 % des tâches |
| DORA, 2024 | +25 % d'usage d'IA : environ +7,2 % d'instabilité |
| Replit, juillet 2025 | Gel explicite. Base de production effacée |
| PocketOS, avril 2026 | Volume de production supprimé. Neuf secondes |
| GitClear, 2025 | Code déplacé : 25 % en 2021, moins de 10 % en 2024 |
| METR, juillet 2025 | 19 % de temps en plus. Prévision : 24 % de moins. Impression d'aller plus vite |
| Godot, 2026 | Contributions d'agent refusées. Complétion gardée |
| GitHub, 1er juin 2026 | Tokens facturés. Une session d'agent coûte plus qu'une question courte |

# Module 3 — Commentaires, complétion, Ctrl+I

Le code qui reste s'écrit avec trois gestes. Toujours dans cet ordre.




- L'algorithme est en commentaires. Dans le fichier. Avant le code.
- La complétion implémente ce qui est écrit.

> **DÉFINITION — COMPLÉTION**
> Proposition au curseur. Elle lit le commentaire voisin. Elle ne choisit pas la découpe.

- Chaque frappe est une requête neuve.
- Ctrl+I ajuste du code déjà écrit. Ctrl+I n'est pas le chat.
- Ce framework fait gagner du temps.

## LES TOUCHES

| TOUCHE | EFFET |
|---|---|
| Tab | Accepter la proposition. |
| Échap | Effacer la proposition. |
| Ctrl+→ | Le mot suivant. Ou la ligne suivante. |
| Cmd+→ | Le même geste, sur Mac. |

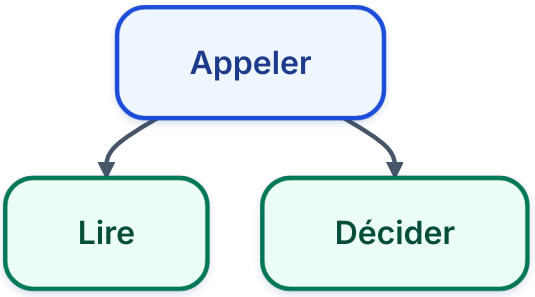
Suggestions inline · Raccourcis clavier

## LE SNOOZE

- Menu Copilot, dans la barre d'état.
- Chaque appui ajoute 5 minutes.
- La complétion se tait pendant ce temps.
- Palette : « Snooze Inline Suggestions ».
- Reprise : « Cancel Snooze Inline Suggestions ».
- Aucun raccourci par défaut.

### LA DÉCOUPE

- Une fonction. Une responsabilité.
- Deux fonctions qui s'enchaînent ne font qu'appeler.
- Elles ne refont pas le travail.




- La fonction qui appelle ne relit pas.
- Elle ne redécide pas.

## Module 4 — Ce qui part dans la requête

**DÉFINITION — TOKEN**

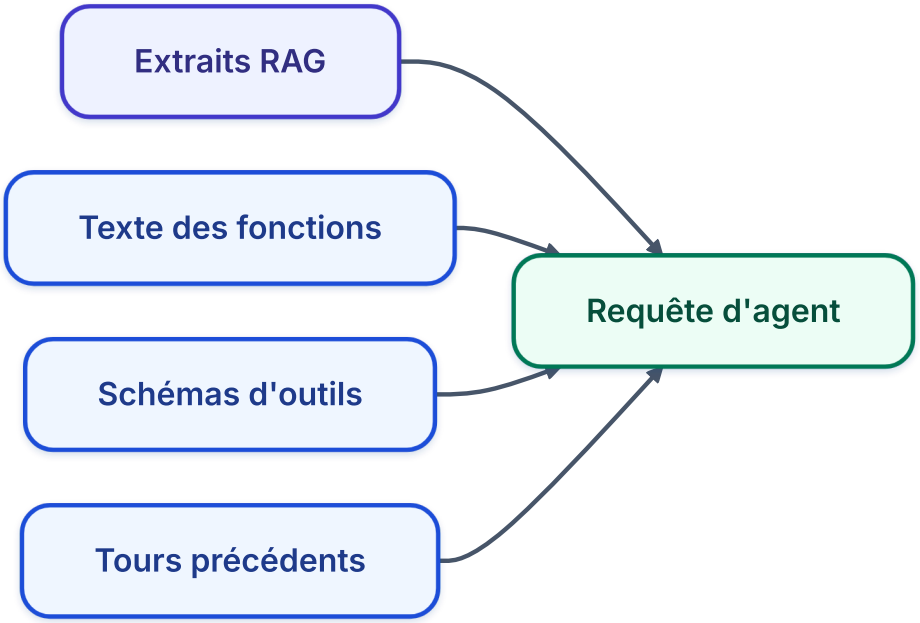
Unité que le modèle lit et produit. Ni un caractère. Ni un mot.

**EXEMPLE**

Ajouter si l'identifiant est absent.

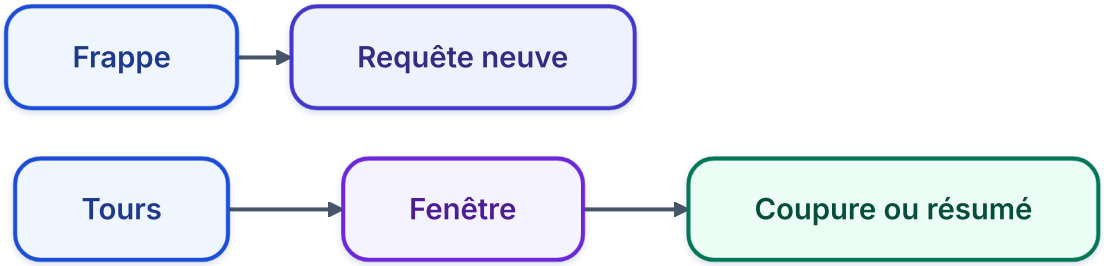
- Pas un token par lettre.
- Pas forcément un token par mot.
- Le découpage dépend du modèle.
- La fenêtre se compte en tokens.
- Chaque élément placé dans la requête consomme des tokens.






## HISTORIQUE

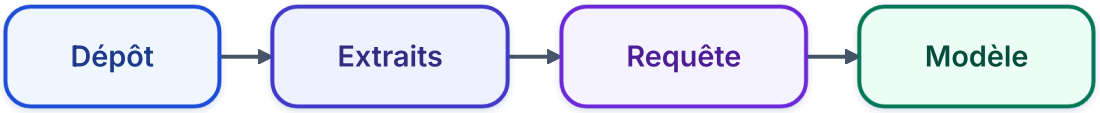
L'historique : les tours renvoyés dans une requête. Ni l'éditeur. Ni les commits.




- La complétion n'a pas d'historique. Chaque frappe est une requête neuve.
- Le commentaire à côté du curseur part dans cette requête.
- L'agent renvoie les tours précédents, tant qu'ils tiennent dans la fenêtre.
- Au-delà, les plus anciens sont coupés ou résumés.

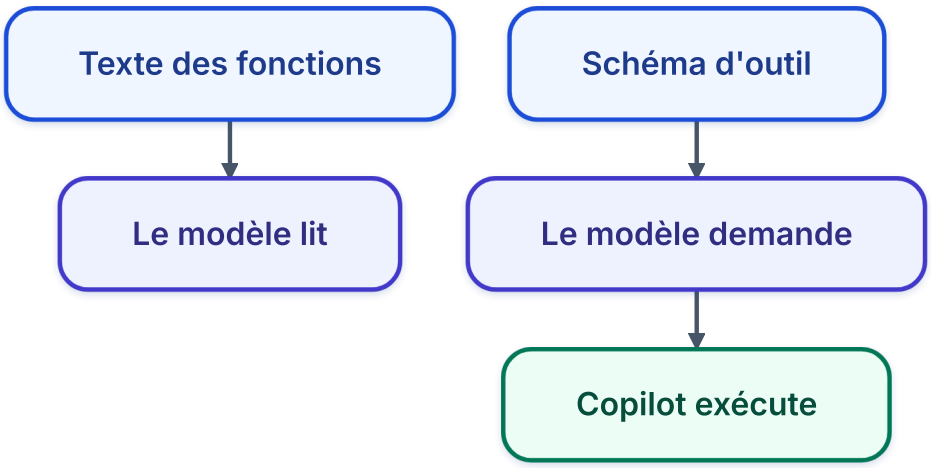
## RAG

> **DÉFINITION — RAG**
> Recherche d'extraits du dépôt. Ces extraits sont collés dans la requête. Le modèle n'a pas le projet en mémoire.




- Le dépôt est cherché. Des extraits sont collés. Le modèle lit ces extraits.
- Un fichier fermé ne part pas parce qu'il est dans le dossier.

### DEUX LISTES

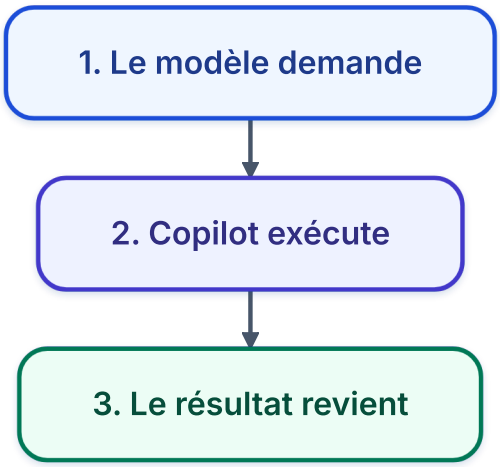



- Une fonction écrite dans le fichier est du texte.
- L'outil qui lance un test est un schéma.
- Le modèle demande le schéma. Copilot l'exécute.

## Module 5 — Qu'est-ce qu'un agent

Un agent est le même middleware, avec une boucle.

- Copilot fabrique la requête.
- Il l'envoie.
- Il réinjecte.
- Puis la boucle recommence.




- Le modèle demande un outil.
- Copilot l'exécute.
- Le résultat revient dans la requête suivante.

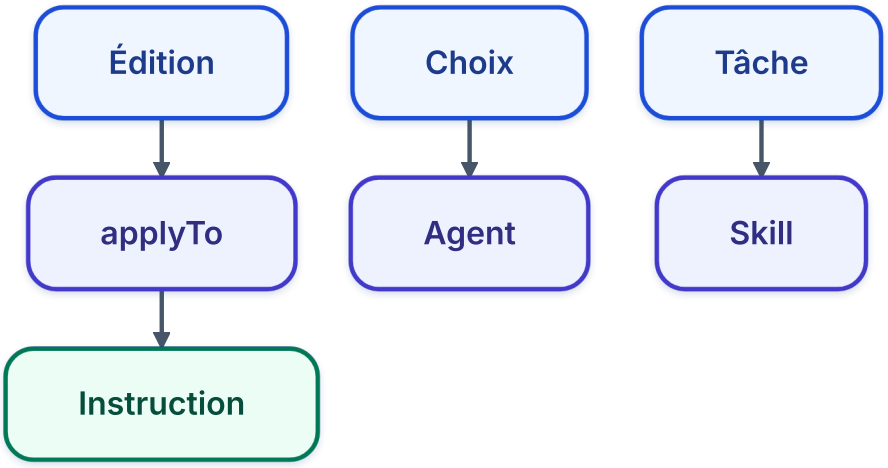
> **DÉFINITION — OUTIL**
> Lire. Chercher. Modifier. Lancer un test.

| OUTIL | EFFET |
| --- | --- |
| Lire | Le texte revient dans la requête suivante |
| Chercher | Les extraits reviennent |
| Modifier | Le changement est appliqué |
| Lancer un test | Le résultat revient |

- Le RAG remet des extraits à chaque tour.
- L'agent n'a pas le projet en mémoire.
- Fenêtre pleine : les tours anciens sont coupés ou résumés.
- L'agent écrit les tests unitaires. L'agent fait un POC.
- Le code qui reste : commentaires, complétion, Ctrl+I.

## Module 6 — Les configurations

Une configuration est un prompt. Il entre dans la requête à un moment précis.




- Les instructions ne partent pas à chaque requête.
- Elles partent quand une édition part.
- Seulement si le fichier touché correspond au `applyTo`.

> **DÉFINITION — APPLYTO**
> Motif de chemin. Fichier `*.instructions.md`. L'instruction part si le fichier correspond.

- Un agent configuré part au moment du choix.
- Il porte le rôle et la liste des outils.
- Il ne part pas avant ce choix.
- Un skill part si la tâche correspond à sa description.
- L'instruction est payée en tokens sur cette édition.
- Le skill est payé seulement s'il est chargé.

| CONFIGURATION | FICHIER |
| --- | --- |
| Instruction | `.github/instructions/` `*.instructions.md` |
| Agent | `.github/agents/` `*.agent.md` |
| Skill | `.github/skills/<nom>/` `SKILL.md` |

## LES EN-TÊTES

- Chaque type a sa liste d'en-têtes.
- La page donne la liste complète.

| TYPE | EN-TÊTES | LISTE |
| --- | --- | --- |
| Instruction | `name` `description` `applyTo` | Liste des en-têtes |
| Agent | `description` `name` `tools` | Liste des en-têtes |
| Skill | `name` `description` | Liste des en-têtes |